

1. Probability & Statistics

```
1 # Import required libraries
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6 from scipy.special import comb, perm, factorial
7 from itertools import permutations, combinations, combinations_with_replacement
8 import pandas as pd
9
10 # Set style for better visualizations
11 sns.set_style("whitegrid")
12 plt.rcParams['figure.figsize'] = (10, 6)
13 plt.rcParams['font.size'] = 11
14
15 print("Libraries imported successfully!")
```

Libraries imported successfully!

Random Choice

```
1 # Simulate 1000 coin flips
2 n_flips = 1000
3 coin_flips = np.random.choice(['Heads', 'Tails'], size=n_flips, p=[0.5, 0.5])
4
5 # Count outcomes
6 unique, counts = np.unique(coin_flips, return_counts=True)
7 results = dict(zip(unique, counts))
8
9 print(results.items())
10
11 print(f"Results from {n_flips} coin flips:")
12 for outcome, count in results.items():
13     print(f" {outcome}: {count} ({count/n_flips*100:.1f}%)")
```

```
dict_items([(np.str_('Heads'), np.int64(482)), (np.str_('Tails'), np.int64(518))])
Results from 1000 coin flips:
Heads: 482 (48.2%)
Tails: 518 (51.8%)
```

Factorial, Permutation, Combination

```
1 # Example 1: Permutations - Arranging letters A, C, G, T
2
3 letters = ['A', 'C', 'G', 'T']
4 n = len(letters)
5
6 # Calculate using formula
7 perm_count = factorial(n)
8 print(f"Number of permutations of {letters}: {int(perm_count)}")
9
10 # Generate all permutations
11 all_perms = list(permutations(letters))
12 all_combs = list(combinations(letters, r=3))
13 print(f"\nFirst 10 permutations:")
14 for i, perm in enumerate(all_perms[:10], 1):
15     print(f" {i}. {''.join(perm)}")
16 print(f"\nFirst 10 combinations:")
17 for i, comb in enumerate(all_combs[:10], 1):
18     print(f" {i}. {''.join(comb)}")
19
20
21 print(f"\nTotal generated: {len(all_perms)} (matches formula: {len(all_perms) == perm_count})")
22 print(f"\nTotal generated: {len(all_combs)}")
```

Number of permutations of ['A', 'C', 'G', 'T']: 24

First 10 permutations:

1. ACGT
2. ACTG
3. AGCT
4. AGTC
5. ATCG
6. ATGC
7. CAGT
8. CATG
9. CGAT

10. CGTA

First 10 combinations:

1. ACG
2. ACT
3. AGT
4. CGT

Total generated: 24 (matches formula: True)

Total generated: 4

```
1 # Example 3: Permutations with repetition
2
3 # How many same-length arrangements of "MACHCHI" are possible?
4 word = "MACHCHI"
5 n_total = len(word)
6
7 # Count frequency of each letter
8 from collections import Counter
9 letter_counts = Counter(word)
10 print(f"Word: {word}")
11 print(f"Letter frequencies: {dict(letter_counts)}")
12
13 # Formula: n! / (k1! × k2! × ... × km!)
14 numerator = factorial(n_total)
15 denominator = 1
16 for count in letter_counts.values():
17     denominator *= factorial(count)
18
19 arrangements = numerator // denominator
20 print(f"\nNumber of arrangements: {n_total}! / (2! × 2! × 2!) = {int(arrangements)}")
```

Word: MACHCHI

Letter frequencies: {'M': 1, 'A': 1, 'C': 2, 'H': 2, 'I': 1}

Number of arrangements: 7! / (2! × 2! × 2!) = 1260

```
1 # Example 4: Combinations - Order doesn't matter
2 from scipy.special import comb, perm, factorial
3 # Choosing a hockey team from 17 players (assume 11 players per team)
4 n_players = 17
5 r_team = 11
6
7 # Using scipy
8 c_nr = comb(n_players, r_team, exact=True)
9 print(f"C({n_players}, {r_team}) = {n_players}! / ({r_team}! × {n_players-r_team}!) = {c_nr}")
10
11 # Example: Choosing 3 ice cream flavors from 6 available
12 n_flavors = 6
13 r_scoops = 3
14 combinations_count = comb(n_flavors, r_scoops, exact=True)
15 print(f"\nWays to choose {r_scoops} ice cream flavors from {n_flavors}: {combinations_count}")
16
17 # Generate actual combinations
18 flavors = ['Vanilla', 'Chocolate', 'Strawberry', 'Mango', 'Pistachio', 'Butterscotch']
19 flavor_combos = list(combinations(flavors, r_scoops))
20 print(f"\nFirst 10 combinations:")
21 for i, combo in enumerate(flavor_combos[:10], 1):
22     print(f" {i}. {' '.join(combo)}")
```

$C(17, 11) = 17! / (11! \times 6!) = 12376$

Ways to choose 3 ice cream flavors from 6: 20.0

First 10 combinations:

1. Vanilla, Chocolate, Strawberry
2. Vanilla, Chocolate, Mango
3. Vanilla, Chocolate, Pistachio
4. Vanilla, Chocolate, Butterscotch
5. Vanilla, Strawberry, Mango
6. Vanilla, Strawberry, Pistachio
7. Vanilla, Strawberry, Butterscotch
8. Vanilla, Mango, Pistachio
9. Vanilla, Mango, Butterscotch
10. Vanilla, Pistachio, Butterscotch

```
1 # Example 5: Combinations with repetition
2
3 # Choosing 3 scoops from 6 flavors WITH repetition allowed
4 n_flavors = 6
5 r_scoops = 3
```

```

6
7 # Formula: C(n+r-1, r)
8 combos_with_rep = comb(n_flavors + r_scoops - 1, r_scoops, exact=True)
9 print(f"Combinations with repetition: C({n_flavors}+{r_scoops}-1, {r_scoops}) = {combos_with_rep}")
10
11 # Generate some examples
12 flavors_short = ['V', 'C', 'S', 'M', 'P', 'B'] # Abbreviated names
13 combos_rep = list(combinations_with_replacement(flavors_short, r_scoops))
14 print(f"\nFirst 10 combinations (with repetition allowed):")
15 for i, combo in enumerate(combos_rep[:10], 1):
16     print(f" {i}. {'-'.join(combo)}")
17
18 print(f"\nTotal: {len(combos_rep)} combinations")

```

Combinations with repetition: C(6+3-1, 3) = 56

First 10 combinations (with repetition allowed):

1. V-V-V
2. V-V-C
3. V-V-S
4. V-V-M
5. V-V-P
6. V-V-B
7. V-C-C
8. V-C-S
9. V-C-M
10. V-C-P

Total: 56 combinations

Probability Formulas

Concept	Formula
Permutations (no rep)	$P(n,r) = n!/(n-r)!$
Permutations (with rep)	n^r
Combinations (no rep)	$C(n,r) = n!/(r!(n-r)!)$
Combinations (with rep)	$C(n+r-1, r)$
Conditional Probability	$P(A B) = P(A \cap B)/P(B)$
Independence	$P(A \cap B) = P(A) \times P(B)$
Bayes' Theorem	$P(A B) = P(B A) \times P(A)/P(B)$

Statistics Formulas

```

1 data = np.random.randint(0,10000,1000)
2
3 # Central tendency
4 mean_val = np.mean(data)
5 median_val = np.median(data)
6 mode_result = stats.mode(data, keepdims=True)
7 print(mode_result)
8 mode_val = mode_result.mode[0] if len(mode_result.mode) > 0 else 'N/A'
9
10 # Dispersion
11 variance = np.var(data, ddof=1)
12 std_dev = np.std(data, ddof=1)
13 range_val = np.max(data) - np.min(data)
14
15 # Quartiles
16 q1 = np.percentile(data, 25)
17 q3 = np.percentile(data, 75)
18 iqr = q3 - q1
19
20 # Other
21 n = len(data)
22 min_val = np.min(data)
23 max_val = np.max(data)

```

ModeResult(mode=array([8700]), count=array([3]))

Hypothesis Tests

```

1 from scipy import stats
2
3 # Create a standard normal distribution
4 x = np.linspace(-4, 4, 1000)
5 y = stats.norm.pdf(x, 0, 1)
6
7 alpha = 0.05
8
9 # Two-sided test

```

```

10 critical_left = stats.norm.ppf(alpha/2)
11 critical_right = stats.norm.ppf(1 - alpha/2)
12
13 # Right-tailed test
14 critical_right_one = stats.norm.ppf(1 - alpha)
15
16 # Left-tailed test
17 critical_left_one = stats.norm.ppf(alpha)
18
19 print("\nCritical Values ( $\alpha = 0.05$ ):")
20 print(f"  Two-sided: [{critical_left:.3f}, {critical_right:.3f}]")
21 print(f"  Right-tailed: {critical_right_one:.3f}")
22 print(f"  Left-tailed: {critical_left_one:.3f}")

```

```

Critical Values ( $\alpha = 0.05$ ):
  Two-sided: [-1.960, 1.960]
  Right-tailed: 1.645
  Left-tailed: -1.645

```

▼ t-test (two tailed)

```

1 # Example: Two-sided test for light bulb lifetime
2 np.random.seed(42)
3
4 # Sample data: 50 light bulbs
5 # True mean is 980 hours (different from claimed 1000)
6 bulb_lifetimes = np.random.normal(loc=980, scale=50, size=50)
7
8 # Hypothesized mean
9 mu_0 = 1000
10
11 # Perform two-sided t-test
12 t_statistic, p_value_two_sided = stats.ttest_1samp(bulb_lifetimes, mu_0, alternative='two-sided')
13
14 print("Two-Sided Hypothesis Test: Light Bulb Lifetime")
15 print("=" * 60)
16 print(f"H0:  $\mu = \{mu\_0\}$  hours")
17 print(f"H1:  $\mu \neq \{mu\_0\}$  hours (two-sided)")
18 print(f"\nSample statistics:")
19 print(f"  Sample size: {len(bulb_lifetimes)}")
20 print(f"  Sample mean: {np.mean(bulb_lifetimes):.2f} hours")
21 print(f"  Sample std dev: {np.std(bulb_lifetimes, ddof=1):.2f} hours")
22 print(f"\nTest results:")
23 print(f"  t-statistic: {t_statistic:.4f}")
24 print(f"  p-value (two-sided): {p_value_two_sided:.4f}")
25 print(f"\nDecision ( $\alpha = 0.05$ ):")
26 if p_value_two_sided < 0.05:
27     print(f"  ✗ Reject H0: The mean lifetime is significantly different from {mu_0} hours")
28     print(f"  → Actual mean appears to be {np.mean(bulb_lifetimes):.1f} hours")
29 else:
30     print(f"  ✓ Fail to reject H0: No significant difference from {mu_0} hours")

```

```

Two-Sided Hypothesis Test: Light Bulb Lifetime
=====
H0:  $\mu = 1000$  hours
H1:  $\mu \neq 1000$  hours (two-sided)

Sample statistics:
  Sample size: 50
  Sample mean: 968.73 hours
  Sample std dev: 46.68 hours

Test results:
  t-statistic: -4.7370
  p-value (two-sided): 0.0000

Decision ( $\alpha = 0.05$ ):
  ✗ Reject H0: The mean lifetime is significantly different from 1000 hours
  → Actual mean appears to be 968.7 hours

```

▼ t-test (right tailed)

```

1 # Example: Right-tailed test for teaching method effectiveness
2 np.random.seed(123)
3
4 # Sample data: 40 students using new teaching method
5 # True mean is 78 (higher than traditional 75)
6 new_method_scores = np.random.normal(loc=78, scale=8, size=40)
7
8 # Hypothesized mean (traditional method)
9 mu_0 = 75

```

```

10
11 # Perform right-tailed t-test
12 t_statistic, p_value_right = stats.ttest_1samp(new_method_scores, mu_0, alternative='greater')
13
14 print("Right-Tailed Hypothesis Test: Teaching Method Effectiveness")
15 print("=" * 60)
16 print(f"H0:  $\mu \leq \{\text{mu\_0}\}$ ")
17 print(f"H1:  $\mu > \{\text{mu\_0}\}$  (right-tailed)")
18 print(f"\nSample statistics:")
19 print(f"   Sample size: {len(new_method_scores)}")
20 print(f"   Sample mean: {np.mean(new_method_scores):.2f}")
21 print(f"   Sample std dev: {np.std(new_method_scores, ddof=1):.2f}")
22 print(f"\nTest results:")
23 print(f"   t-statistic: {t_statistic:.4f}")
24 print(f"   p-value (right-tailed): {p_value_right:.4f}")
25 print(f"\nDecision ( $\alpha = 0.05$ ):")
26 if p_value_right < 0.05:
27     print(f"   X Reject H0: New method produces significantly higher scores")
28     print(f"   → Mean score with new method: {np.mean(new_method_scores):.1f} vs traditional: {mu_0}")
29 else:
30     print(f"   ✓ Fail to reject H0: No evidence that new method is better")

```

```

Right-Tailed Hypothesis Test: Teaching Method Effectiveness
=====
H0:  $\mu \leq 75$ 
H1:  $\mu > 75$  (right-tailed)

Sample statistics:
  Sample size: 40
  Sample mean: 77.33
  Sample std dev: 9.45

Test results:
  t-statistic: 1.5616
  p-value (right-tailed): 0.0632

Decision ( $\alpha = 0.05$ ):
  ✓ Fail to reject H0: No evidence that new method is better

```

✓ t-test (left tailed)

```

1 # Example: Left-tailed test for production time
2 np.random.seed(456)
3
4 # Sample data: 35 production runs with new process
5 # True mean is 115 minutes (faster than old 120)
6 new_process_times = np.random.normal(loc=115, scale=10, size=35)
7
8 # Hypothesized mean (old process)
9 mu_0 = 120
10
11 # Perform left-tailed t-test
12 t_statistic, p_value_left = stats.ttest_1samp(new_process_times, mu_0, alternative='less')
13
14 print("Left-Tailed Hypothesis Test: Production Time Reduction")
15 print("=" * 60)
16 print(f"H0:  $\mu \geq \{\text{mu\_0}\}$  minutes")
17 print(f"H1:  $\mu < \{\text{mu\_0}\}$  minutes (left-tailed)")
18 print(f"\nSample statistics:")
19 print(f"   Sample size: {len(new_process_times)}")
20 print(f"   Sample mean: {np.mean(new_process_times):.2f} minutes")
21 print(f"   Sample std dev: {np.std(new_process_times, ddof=1):.2f} minutes")
22 print(f"\nTest results:")
23 print(f"   t-statistic: {t_statistic:.4f}")
24 print(f"   p-value (left-tailed): {p_value_left:.4f}")
25 print(f"\nDecision ( $\alpha = 0.05$ ):")
26 if p_value_left < 0.05:
27     print(f"   X Reject H0: New process is significantly faster")
28     time_saved = mu_0 - np.mean(new_process_times)
29     print(f"   → Time saved: {time_saved:.1f} minutes per production run")
30 else:
31     print(f"   ✓ Fail to reject H0: No evidence that new process is faster")

```

```

Left-Tailed Hypothesis Test: Production Time Reduction
=====
H0:  $\mu \geq 120$  minutes
H1:  $\mu < 120$  minutes (left-tailed)

Sample statistics:
  Sample size: 35
  Sample mean: 116.46 minutes
  Sample std dev: 9.89 minutes

```

```
Test results:
t-statistic: -2.1209
p-value (left-tailed): 0.0207

Decision ( $\alpha = 0.05$ ):
X Reject  $H_0$ : New process is significantly faster
→ Time saved: 3.5 minutes per production run
```

▼ Comparing One-Sided and Two-Sided Tests

Key Differences:

Aspect	Two-Sided	Right-Tailed	Left-Tailed
H_1	$\mu \neq \mu_0$	$\mu > \mu_0$	$\mu < \mu_0$
Rejection Region	Both tails	Right tail only	Left tail only
α split	$\alpha/2$ each side	α on right	α on left
Use when	Testing for any difference	Testing for increase	Testing for decrease
p-value	$P(T \geq t)$	$P(T \geq t)$	$P(T \leq t)$

Choosing the Right Test:

- Use **two-sided** when you don't have a directional hypothesis (most common in research)
- Use **right-tailed** when you specifically want to prove something is greater
- Use **left-tailed** when you specifically want to prove something is less

Important: One-sided tests have more power to detect effects in the specified direction, but they cannot detect effects in the opposite direction!

```
1 # Compare all three tests on the same data
2 np.random.seed(789)
3
4 # Sample data
5 sample_data = np.random.normal(loc=105, scale=15, size=30)
6 mu_0 = 100
7
8 # Perform all three tests
9 t_stat, p_two = stats.ttest_1samp(sample_data, mu_0, alternative='two-sided')
10 _, p_right = stats.ttest_1samp(sample_data, mu_0, alternative='greater')
11 _, p_left = stats.ttest_1samp(sample_data, mu_0, alternative='less')
12
13 print("Comparison of Hypothesis Tests on Same Data")
14 print("=" * 60)
15 print(f"Sample mean: {np.mean(sample_data):.2f}")
16 print(f"Hypothesized mean: {mu_0}")
17 print(f"t-statistic: {t_stat:.4f}")
18 print(f"\nP-values:")
19 print(f"  Two-sided ( $H_1: \mu \neq \{mu\_0\}$ ): p = {p_two:.4f}")
20 print(f"  Right-tailed ( $H_1: \mu > \{mu\_0\}$ ): p = {p_right:.4f}")
21 print(f"  Left-tailed ( $H_1: \mu < \{mu\_0\}$ ): p = {p_left:.4f}")
22 print(f"\nNote: p_two ≈ 2 × min(p_right, p_left)")
23 print(f"Verification: 2 × {min(p_right, p_left):.4f} = {2*min(p_right, p_left):.4f}")
24
25 # Decision at  $\alpha = 0.05$ 
26 print(f"\nDecisions at  $\alpha = 0.05$ :")
27 print(f"  Two-sided: {'Reject H_0' if p_two < 0.05 else 'Fail to reject H_0'}")
28 print(f"  Right-tailed: {'Reject H_0' if p_right < 0.05 else 'Fail to reject H_0'}")
29 print(f"  Left-tailed: {'Reject H_0' if p_left < 0.05 else 'Fail to reject H_0'}")
```

```
Comparison of Hypothesis Tests on Same Data
=====
Sample mean: 104.30
Hypothesized mean: 100
t-statistic: 1.7852

P-values:
  Two-sided ( $H_1: \mu \neq 100$ ): p = 0.0847
  Right-tailed ( $H_1: \mu > 100$ ): p = 0.0423
  Left-tailed ( $H_1: \mu < 100$ ): p = 0.9577

Note: p_two ≈ 2 × min(p_right, p_left)
Verification: 2 × 0.0423 = 0.0847

Decisions at  $\alpha = 0.05$ :
  Two-sided: Fail to reject  $H_0$ 
  Right-tailed: Reject  $H_0$ 
  Left-tailed: Fail to reject  $H_0$ 
```

▼ 2. Optimization Methods

```

1 # Import necessary libraries
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.optimize import minimize, LinearConstraint, Bounds
5 from scipy.optimize import linprog, minimize_scalar
6 from scipy.optimize import fminbound, brent
7 from scipy.optimize import least_squares
8 import warnings
9 warnings.filterwarnings('ignore')
10
11 # Set random seed for reproducibility
12 np.random.seed(42)
13
14 # Configure plotting
15 plt.style.use('seaborn-v0_8-darkgrid')
16 plt.rcParams['figure.figsize'] = (12, 6)
17 plt.rcParams['font.size'] = 10

```

▼ Single Variable Unconstrained Optimization

```

1 # This is a scipy method for finding the minimizer of an Univariate objective function
2 from scipy.optimize import minimize_scalar
3 result = minimize_scalar(lambda x: -profit_function(x), bounds=(0, 5000), method='bounded')
4 x_optimal_numerical = result.x
5 f_optimal_numerical = -result.fun

```

▼ Multi Variable Unconstrained Optimization

```

1 # This is a scipy method for finding the minimizer of an Multivariate objective function
2 from scipy.optimize import minimize
3 result = minimize(lambda x: -profit_2d(x), x0=np.array([0, 0]), method='BFGS')
4 x_optimal_numerical = result.x
5 f_optimal_numerical = -result.fun

```

```

1 from scipy.optimize import minimize
2 # Optimize Sphere function
3 print("1. SPHERE FUNCTION:  $f(x) = \sum x_i^2$ ")
4 print("-" * 40)
5 x0_sphere = np.array([5.0, 5.0, 5.0])
6 result_sphere = minimize(sphere_function, x0_sphere, method='BFGS')
7 print(f"Initial point: {x0_sphere}")
8 print(f"Optimal point: {result_sphere.x}")
9 print(f"Optimal value: {result_sphere.fun:.2e}")
10 print(f"Success: {result_sphere.success}")
11 print(f"Iterations: {result_sphere.nit}")
12 print()

```

```

1 from scipy.optimize import minimize
2 # Optimize Rosenbrock function
3 print("2. ROSEN BROCK FUNCTION:  $f(x) = \sum [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2]$ ")
4 print("-" * 40)
5 x0_rosenbrock = np.array([-1.2, 1.0])
6 result_rosenbrock = minimize(rosenbrock_function, x0_rosenbrock, method='BFGS')
7 print(f"Initial point: {x0_rosenbrock}")
8 print(f"Optimal point: {result_rosenbrock.x}")
9 print(f"Optimal value: {result_rosenbrock.fun:.2e}")
10 print(f"Success: {result_rosenbrock.success}")
11 print(f"Iterations: {result_rosenbrock.nit}")
12 print()

```

```

1 from scipy.optimize import minimize
2 # Optimize Rastrigin function (multimodal)
3 print("3. RASTRIGIN FUNCTION:  $f(x) = 10n + \sum [x_i^2 - 10\cos(2\pi x_i)]$ ")
4 print("-" * 40)
5 x0_rastrigin = np.array([2.0, 2.0])
6 result_rastrigin = minimize(rastrigin_function, x0_rastrigin, method='L-BFGS-B')
7 print(f"Initial point: {x0_rastrigin}")
8 print(f"Optimal point: {result_rastrigin.x}")
9 print(f"Optimal value: {result_rastrigin.fun:.2e}")
10 print(f"Success: {result_rastrigin.success}")
11 print(f"Iterations: {result_rastrigin.nit}")
12 print(f>Note: Rastrigin is highly multimodal; local optimum found.")
13 print()

```

✓ Quadratic Programming

```
1 from scipy.optimize import minimize
2 # Problem: Portfolio optimization
3 # Minimize: (1/2) x^T Q x - r^T x (minimize risk - expected return)
4 # Subject to: Σ x_i = 1 (budget constraint), x_i ≥ 0
5
6 # Covariance matrix (risk)
7 Q = np.array([[4, 1, 0.5],
8               [1, 3, 0.2],
9               [0.5, 0.2, 2]])
10
11 # Expected returns
12 r = np.array([0.05, 0.08, 0.06])
13
14 # Objective: minimize (1/2) x^T Q x - r^T x
15 def qp_objective(x):
16     return 0.5 * x @ Q @ x - r @ x
17
18 def qp_gradient(x):
19     return Q @ x - r
20
21 # Constraints: Σ x_i = 1, x_i ≥ 0
22 constraints = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1})
23 bounds = [(0, 1) for _ in range(3)]
24
25 # Initial guess
26 x0 = np.array([0.33, 0.33, 0.34])
27
28 # Solve QP
29 result_qp = minimize(qp_objective, x0, method='SLSQP', bounds=bounds, constraints=constraints)
```

✓ Non-Linear Programming

```
1 from scipy.optimize import minimize
2 # Constraints
3 constraints = {'type': 'eq', 'fun': nlp_constraint}
4
5 # Try different initial points
6 initial_points = [np.array([1.0, 1.0]), np.array([0.5, 1.5]), np.array([-1.0, -1.0])]
7 result = minimize(nlp_objective, x0, method='SLSQP', constraints=constraints)
```

✓ 3. Method of Moments

✓ Exponential Distribution

Problem: Given data $x_1, x_2, \dots, x_n$ from $\text{Exp}(\lambda)$, estimate λ .

Exponential Distribution:

- PDF: $f(x|\lambda) = \lambda e^{-\lambda x}$ for $x \geq 0$
- Mean: $E[X] = \frac{1}{\lambda}$

Method of Moments:

1. First sample moment: $m_1 = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
2. First theoretical moment: $\mu_1 = E[X] = \frac{1}{\lambda}$
3. Set equal: $\bar{x} = \frac{1}{\lambda}$
4. Solve: $\hat{\lambda}_{MoM} = \frac{1}{\bar{x}} = \frac{n}{\sum_{i=1}^n x_i}$

```
1 # Example: Exponential Distribution - Method of Moments
2 import numpy as np
3 # True parameter
4 lambda_true = 2.0
5
6 # Generate sample data
7 n = 100
8 data_exp = np.random.exponential(scale=1/lambda_true, size=n)
9
10 # Method of Moments estimator
11 lambda_mom = 1 / np.mean(data_exp)
12
13 print("Exponential Distribution - Method of Moments")
14 print("=" * 44)
15 print(f"True parameter: λ = {lambda_true}")
```

```

16 print(f"Sample size: n = {n}")
17 print(f"Sample mean: {np.mean(data_exp):.4f}")
18 print(f"\nMoM Estimate: λ̂ = {lambda_mom:.4f}")
19 print(f"Estimation error: {abs(lambda_mom - lambda_true):.4f}")

```

```

Exponential Distribution - Method of Moments
=====
True parameter: λ = 2.0
Sample size: n = 100
Sample mean: 0.5183

MoM Estimate: λ̂ = 1.9295
Estimation error: 0.0705

```

✓ Normal Distribution

Problem: Given data $x_1, x_2, \dots, x_n$ from $N(\mu, \sigma^2)$, estimate μ and σ^2 .

Normal Distribution:

- Mean: $E[X] = \mu$
- Variance: $\text{Var}(X) = E[X^2] - (E[X])^2 = \sigma^2$

Method of Moments:

1. First moment: $m_1 = \bar{x} = \mu \rightarrow \hat{\mu}_{MoM} = \bar{x}$
2. Second moment: $m_2 = \frac{1}{n} \sum x_i^2 = \sigma^2 + \mu^2$
3. Solve: $\hat{\sigma}_{MoM}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$

```

1 # Example: Normal Distribution - Method of Moments
2
3 # True parameters
4 mu_true = 50
5 sigma_true = 10
6
7 # Generate sample data
8 n = 200
9 data_normal = np.random.normal(loc=mu_true, scale=sigma_true, size=n)
10
11 # Method of Moments estimators
12 mu_mom = np.mean(data_normal)
13 sigma2_mom = np.mean((data_normal - mu_mom)**2) # Biased estimator
14 sigma_mom = np.sqrt(sigma2_mom)
15
16 print("Normal Distribution - Method of Moments")
17 print("=" * 39)
18 print(f"True parameters: μ = {mu_true}, σ = {sigma_true}")
19 print(f"Sample size: n = {n}")
20 print(f"\nMoM Estimates:")
21 print(f"  μ̂ = {mu_mom:.4f} (error: {abs(mu_mom - mu_true):.4f})")
22 print(f"  σ̂ = {sigma_mom:.4f} (error: {abs(sigma_mom - sigma_true):.4f})")

```

```

Normal Distribution - Method of Moments
=====
True parameters: μ = 50, σ = 10
Sample size: n = 200

MoM Estimates:
  μ̂ = 49.8563 (error: 0.1437)
  σ̂ = 10.1639 (error: 0.1639)

```

✓ Bernoulli Distribution

Problem: Given binary data $x_1, x_2, \dots, x_n$ from Bernoulli(p), estimate p .

Bernoulli Distribution:

- PMF: $P(X = x) = p^x(1 - p)^{1-x}$ for $x \in \{0, 1\}$
- Mean: $E[X] = p$

Method of Moments:

- Set $\bar{x} = p$
- $\hat{p}_{MoM} = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$

This is also the Maximum Likelihood Estimator (MLE) for the Bernoulli distribution!

```

1 # Example: Bernoulli Distribution - Method of Moments
2
3 # True parameter
4 p_true = 0.7
5
6 # Generate sample data

```

```

7 n = 100
8 data_bernoulli = np.random.binomial(1, p_true, size=n)
9
10 # Method of Moments estimator
11 p_mom = np.mean(data_bernoulli)
12
13 print("Bernoulli Distribution - Method of Moments")
14 print("=" * 42)
15 print(f"True parameter: p = {p_true}")
16 print(f"Sample size: n = {n}")
17 print(f"Number of successes: {np.sum(data_bernoulli)}")
18 print(f"Number of failures: {n - np.sum(data_bernoulli)}")
19 print(f"\nMoM Estimate:  $\hat{p}$  = {p_mom:.4f}")
20 print(f"Estimation error: {abs(p_mom - p_true):.4f}")

```

```

Bernoulli Distribution - Method of Moments
=====
True parameter: p = 0.7
Sample size: n = 100
Number of successes: 70
Number of failures: 30

MoM Estimate:  $\hat{p}$  = 0.7000
Estimation error: 0.0000

```

Uniform Distribution

1

4. Bootstrap

Standard Error

```

1 # Example: Bootstrap Standard Error for the Mean
2 import numpy as np
3 from scipy import stats
4 # Original sample
5 np.random.seed(42)
6 original_sample = np.random.normal(loc=100, scale=15, size=50)
7
8 # Bootstrap parameters
9 B = 10000 # Number of bootstrap samples
10 n = len(original_sample)
11
12 # Perform bootstrap
13 bootstrap_means = []
14 for b in range(B):
15     # Resample with replacement
16     bootstrap_sample = np.random.choice(original_sample, size=n, replace=True)
17     # Compute statistic (mean)
18     bootstrap_means.append(np.mean(bootstrap_sample))
19
20 bootstrap_means = np.array(bootstrap_means)
21
22 # Bootstrap standard error
23 se_boot = np.std(bootstrap_means, ddof=1)
24
25 # Theoretical standard error (for comparison)
26 se_theory = np.std(original_sample, ddof=1) / np.sqrt(n)
27
28 print("Bootstrap Standard Error Estimation")
29 print("=" * 35)
30 print(f"Original sample size: n = {n}")
31 print(f"Original sample mean: {np.mean(original_sample):.4f}")
32 print(f"Original sample std: {np.std(original_sample, ddof=1):.4f}")
33 print(f"Number of bootstrap samples: B = {B}")
34 print(f"\nBootstrap standard error: {se_boot:.4f}")
35 print(f"Theoretical standard error: {se_theory:.4f}")
36 print(f"Difference: {abs(se_boot - se_theory):.4f}")

```

```

Bootstrap Standard Error Estimation
=====
Original sample size: n = 50
Original sample mean: 96.6179
Original sample std: 14.0050
Number of bootstrap samples: B = 10000

Bootstrap standard error: 1.9614
Theoretical standard error: 1.9806
Difference: 0.0192

```

```

1 # Example: Bootstrap Confidence Intervals
2
3 # Using the bootstrap means from previous example
4 alpha = 0.05 # 95% confidence level
5
6 # Method 1: Percentile method
7 ci_percentile = np.percentile(bootstrap_means, [100*alpha/2, 100*(1-alpha/2)])
8
9 # Method 2: Normal method
10 z_critical = stats.norm.ppf(1 - alpha/2)
11 ci_normal = [np.mean(original_sample) - z_critical * se_boot,
12             np.mean(original_sample) + z_critical * se_boot]
13
14 # Theoretical CI (for comparison)
15 ci_theory = stats.t.interval(1-alpha, df=n-1, loc=np.mean(original_sample),
16                             scale=se_theory)
17
18 print("Bootstrap Confidence Intervals (95%)")
19 print("=" * 36)
20 print(f"Point estimate: {np.mean(original_sample):.4f}")
21 print(f"\n1. Percentile Method:")
22 print(f"   [{ci_percentile[0]:.4f}, {ci_percentile[1]:.4f}]")
23 print(f"   Width: {ci_percentile[1] - ci_percentile[0]:.4f}")
24 print(f"\n2. Normal Method:")
25 print(f"   [{ci_normal[0]:.4f}, {ci_normal[1]:.4f}]")
26 print(f"   Width: {ci_normal[1] - ci_normal[0]:.4f}")
27 print(f"\n3. Theoretical t-interval (for comparison):")
28 print(f"   [{ci_theory[0]:.4f}, {ci_theory[1]:.4f}]")
29 print(f"   Width: {ci_theory[1] - ci_theory[0]:.4f}")

```

Bootstrap Confidence Intervals (95%)
 =====
 Point estimate: 96.6179

1. Percentile Method:
 [92.7539, 100.4578]
 Width: 7.7039
2. Normal Method:
 [92.7736, 100.4622]
 Width: 7.6886
3. Theoretical t-interval (for comparison):
 [92.6377, 100.5981]
 Width: 7.9604

```

1 # Example: Bootstrap for Median
2
3 # Generate skewed data (median is more robust than mean for skewed data)
4 np.random.seed(123)
5 skewed_data = np.random.exponential(scale=10, size=60)
6
7 # Bootstrap for median
8 B = 10000
9 n = len(skewed_data)
10 bootstrap_medians = []
11
12 for b in range(B):
13     bootstrap_sample = np.random.choice(skewed_data, size=n, replace=True)
14     bootstrap_medians.append(np.median(bootstrap_sample))
15
16 bootstrap_medians = np.array(bootstrap_medians)
17
18 # Bootstrap standard error and CI
19 se_median = np.std(bootstrap_medians, ddof=1)
20 ci_median = np.percentile(bootstrap_medians, [2.5, 97.5])
21
22 print("Bootstrap for Median (Skewed Data)")
23 print("=" * 60)
24 print(f"Sample size: n = {n}")
25 print(f"Sample mean: {np.mean(skewed_data):.4f}")
26 print(f"Sample median: {np.median(skewed_data):.4f}")
27 print(f"\nBootstrap results:")
28 print(f"  Bootstrap SE(median): {se_median:.4f}")
29 print(f"  95% CI for median: [{ci_median[0]:.4f}, {ci_median[1]:.4f}]")

```

Bootstrap for Median (Skewed Data)
 =====
 Sample size: n = 60
 Sample mean: 9.1027
 Sample median: 6.7019

Bootstrap results:

Bootstrap SE(median): 0.7888
95% CI for median: [5.5010, 8.0143]

```
1 # Comparison: Bootstrap vs Theory for Sample Mean
2
3 # Simulation parameters
4 true_mu = 50
5 true_sigma = 10
6 sample_size = 30
7 n_simulations = 1000
8 B = 5000
9
10 # Storage
11 coverage_theory = 0
12 coverage_bootstrap = 0
13 widths_theory = []
14 widths_bootstrap = []
15
16 np.random.seed(789)
17
18 for sim in range(n_simulations):
19     # Generate sample
20     sample = np.random.normal(true_mu, true_sigma, sample_size)
21
22     # Theoretical CI
23     se_theory = np.std(sample, ddof=1) / np.sqrt(sample_size)
24     ci_theory = stats.t.interval(0.95, df=sample_size-1, loc=np.mean(sample), scale=se_theory)
25     widths_theory.append(ci_theory[1] - ci_theory[0])
26     if ci_theory[0] <= true_mu <= ci_theory[1]:
27         coverage_theory += 1
28
29     # Bootstrap CI
30     bootstrap_means = []
31     for b in range(B):
32         boot_sample = np.random.choice(sample, size=sample_size, replace=True)
33         bootstrap_means.append(np.mean(boot_sample))
34     ci_boot = np.percentile(bootstrap_means, [2.5, 97.5])
35     widths_bootstrap.append(ci_boot[1] - ci_boot[0])
36     if ci_boot[0] <= true_mu <= ci_boot[1]:
37         coverage_bootstrap += 1
38
39 # Results
40 print("Bootstrap vs Theoretical Comparison")
41 print("=" * 60)
42 print(f"True parameters:  $\mu = \{true\_mu\}$ ,  $\sigma = \{true\_sigma\}$ ")
43 print(f"Sample size:  $n = \{sample\_size\}$ ")
44 print(f"Number of simulations:  $\{n\_simulations\}$ ")
45 print(f"Bootstrap resamples per simulation:  $B = \{B\}$ ")
46 print(f"\nCoverage (should be  $\approx 95\%$ ):")
47 print(f"  Theoretical t-interval:  $\{coverage\_theory/n\_simulations*100:.2f\}\%$ ")
48 print(f"  Bootstrap percentile:  $\{coverage\_bootstrap/n\_simulations*100:.2f\}\%$ ")
49 print(f"\nAverage CI width:")
50 print(f"  Theoretical:  $\{np.mean(widths\_theory):.4f\}$ ")
51 print(f"  Bootstrap:  $\{np.mean(widths\_bootstrap):.4f\}$ ")
```

```
Bootstrap vs Theoretical Comparison
=====
True parameters:  $\mu = 50$ ,  $\sigma = 10$ 
Sample size:  $n = 30$ 
Number of simulations: 1000
Bootstrap resamples per simulation:  $B = 5000$ 

Coverage (should be  $\approx 95\%$ ):
  Theoretical t-interval: 94.80%
  Bootstrap percentile: 93.40%

Average CI width:
  Theoretical: 7.3839
  Bootstrap: 6.9408
```

5. Method of Moments v/s Bootstrap

Aspect	Method of Moments	Bootstrap
Type	Parametric	Non-parametric
Assumptions	Requires distributional form	No distributional assumptions
Purpose	Point estimation of parameters	Uncertainty quantification (SE, CI)
Computation	Analytical (usually simple)	Computational (intensive)
Sample Size	Works for any size	Better with larger samples
Flexibility	Limited to moments	Works for any statistic
Efficiency	May not be optimal	Approaches optimal with large B

6. PCA

Math Foundation of PCA

Variance and Covariance

Variance measures the spread of a single variable:

$$\text{Var}(X) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Covariance measures how two variables vary together:

$$\text{Cov}(X, Y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Covariance Matrix for d features:

$$\Sigma = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_d) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots & \text{Cov}(X_2, X_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_d, X_1) & \text{Cov}(X_d, X_2) & \cdots & \text{Var}(X_d) \end{bmatrix}$$

Eigenvalues and Eigenvectors

For a square matrix A and vector v :

$$Av = \lambda v$$

Where:

- v is an **eigenvector** (direction that doesn't change under transformation)
- λ is an **eigenvalue** (scaling factor in that direction)

Key Insight: The eigenvectors of the covariance matrix are the principal components, and the eigenvalues represent the variance explained by each component.

Step-by-Step PCA Algorithm

The PCA Algorithm

Input: Data matrix X of shape (n, d) where n = samples, d = features

Steps:

1. **Standardize the data** (mean = 0, std = 1 for each feature)

$$X_{\text{std}} = \frac{X - \mu}{\sigma}$$

2. **Compute the covariance matrix**

$$\Sigma = \frac{1}{n-1} X_{\text{std}}^T X_{\text{std}}$$

3. **Compute eigenvalues and eigenvectors** of Σ

$$\Sigma v_i = \lambda_i v_i$$

4. **Sort eigenvectors** by eigenvalues in descending order
5. **Select top k eigenvectors** to form projection matrix W
6. **Transform the data**

$$X_{\text{PCA}} = X_{\text{std}} \cdot W$$

Output: Reduced data matrix of shape (n, k)

Why Standardization?

Features with larger scales dominate the variance. Standardization ensures all features contribute equally.

```
1 # Import required libraries
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from mpl_toolkits.mplot3d import Axes3D
```

```

5 import seaborn as sns
6 from sklearn.decomposition import PCA
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.datasets import load_iris, load_digits, make_classification
9 import pandas as pd
10
11 # Set style for better visualizations
12 plt.style.use('seaborn-v0_8-darkgrid')
13 sns.set_palette("husl")
14 np.random.seed(42)
15

```

Libraries imported successfully!

```

1 # Demonstration: Covariance and correlation
2 # Create correlated 2D data
3 np.random.seed(42)
4 mean = [0, 0]
5 cov = [[3, 2.5], # Covariance matrix: variance of X=3, covariance=2.5
6        [2.5, 3]] # variance of Y=3
7 data_2d = np.random.multivariate_normal(mean, cov, 500)
8
9 # Calculate covariance matrix
10 cov_matrix = np.cov(data_2d.T)
11
12 print("Covariance Matrix:")
13 print(cov_matrix)
14 print(f"\nVariance of X: {cov_matrix[0, 0]:.2f}")
15 print(f"\nVariance of Y: {cov_matrix[1, 1]:.2f}")
16 print(f"Covariance(X, Y): {cov_matrix[0, 1]:.2f}")
17 print(f"\nPositive covariance means X and Y tend to increase together")
18
19
20 # Demonstration: Eigenvectors and eigenvalues
21 # Compute eigenvectors and eigenvalues of the covariance matrix
22 eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
23
24 print("\nEigenvalues (variance explained by each component):")
25 for i, eigenval in enumerate(eigenvalues):
26     print(f" PC{i+1}: {eigenval:.3f}")
27
28 print("\nEigenvectors (principal component directions):")
29 for i, eigenvec in enumerate(eigenvectors.T):
30     print(f" PC{i+1}: [{eigenvec[0]:.3f}, {eigenvec[1]:.3f}]")

```

```

Covariance Matrix:
[[2.77573069 2.32567908]
 [2.32567908 2.85962503]]

```

```

Variance of X: 2.78
Variance of Y: 2.86
Covariance(X, Y): 2.33

```

Positive covariance means X and Y tend to increase together

```

Eigenvalues (variance explained by each component):
PC1: 0.492
PC2: 5.144

```

```

Eigenvectors (principal component directions):
PC1: [-0.713, 0.701]
PC2: [-0.701, -0.713]

```

```

1 # Apply PCA using scikit-learn
2 # Standardize first
3 scaler = StandardScaler()
4 X_iris_scaled = scaler.fit_transform(X_iris)
5
6 # Apply PCA to reduce from 4D to 2D
7 pca_sklearn = PCA(n_components=2)
8 X_iris_pca = pca_sklearn.fit_transform(X_iris_scaled)
9
10 print("PCA with Scikit-learn")
11 print("=" * 60)
12 print(f"Original shape: {X_iris.shape}")
13 print(f"Transformed shape: {X_iris_pca.shape}")
14 print(f"\nExplained variance ratio:")
15 for i, var in enumerate(pca_sklearn.explained_variance_ratio_):
16     print(f" PC{i+1}: {var:.4f} ({var*100:.2f}%)")
17 print(f"\nTotal variance explained: {np.sum(pca_sklearn.explained_variance_ratio_):.4f} ({np.sum(pca_sklearn.explained_variance_ratio_)*100:.2f}%)")
18
19 print(f"\nPrincipal components (loadings):")
20 components_df = pd.DataFrame(
21     pca_sklearn.components_,
22     columns=feature_names,

```

```
23     index=['PC1', 'PC2']
24 )
25 print(components_df)
```

Limitations of PCA

1. **Linearity**: PCA only captures linear relationships
 - Solution: Use kernel PCA or other nonlinear methods (t-SNE, UMAP)
2. **Variance ≠ Information**: High variance doesn't always mean high importance
 - Example: Outliers can create high-variance components that are just noise
3. **Interpretability**: Principal components are linear combinations of all features
 - Hard to interpret what each PC "means"
4. **Scaling Sensitivity**: Results depend heavily on feature scaling
 - Always standardize before PCA
5. **Assumes Gaussian Distribution**: Works best when data is approximately normal

When NOT to Use PCA

- When interpretability of original features is critical
- When relationships are highly nonlinear
- When you have very few features already
- When features have very different meanings (e.g., mixing categorical and continuous)

Best Practices

1. **Always standardize** your data before PCA
2. **Check explained variance** to choose appropriate k
3. **Visualize** the results (scree plot, biplots)
4. **Consider alternatives** for nonlinear data (kernel PCA, t-SNE, UMAP)
5. **Validate** on downstream tasks (classification, clustering, etc.)

7. Clustering

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.cluster import KMeans
6 from sklearn.datasets import make_blobs, load_iris, load_wine
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import silhouette_score, davies_bouldin_score
9 from scipy.spatial.distance import cdist
```

K-Means

```
1 # Generate data with clear clusters
2 X_elbow, y_elbow = make_blobs(n_samples=300, centers=4, n_features=2,
3                               cluster_std=0.7, random_state=42)
4
5 # Calculate WCSS for different k values
6 k_range = range(1, 11)
7 wcss = []
8 silhouette_scores = []
9
10 for k in k_range:
11     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
12     kmeans.fit(X_elbow)
13     wcss.append(kmeans.inertia_)
14
15     # Silhouette score (only for k >= 2)
16     if k >= 2:
17         silhouette_scores.append(silhouette_score(X_elbow, kmeans.labels_))
18     else:
19         silhouette_scores.append(0)
20
21 print("\nElbow Method Results:")
```

```

22 print("="*21)
23 for k, w, s in zip(k_range, wcss, silhouette_scores):
24     print(f"{k:2d}: {wcss: 8.2f}; Silhouette = {s: .4f}")

```

Elbow Method Results:

=====

```

k = 1; WCSS = 19710.62; Silhouette = 0.0000
k = 2; WCSS = 9125.81; Silhouette = 0.6093
k = 3; WCSS = 1840.87; Silhouette = 0.7888
k = 4; WCSS = 277.52; Silhouette = 0.8546
k = 5; WCSS = 251.00; Silhouette = 0.7394
k = 6; WCSS = 225.56; Silhouette = 0.5993
k = 7; WCSS = 200.38; Silhouette = 0.4571
k = 8; WCSS = 178.17; Silhouette = 0.3296
k = 9; WCSS = 160.50; Silhouette = 0.3379
k = 10; WCSS = 142.20; Silhouette = 0.3657

```

```

1 kmeans = KMeans(n_clusters=4, n_init=10)
2 kmeans.fit(X_elbow)
3 print("WCSS =",np.round(kmeans.inertia_,2))
4 print("Silhouette score=",np.round(silhouette_score(X_elbow, kmeans.labels_),2))

```

```

WCSS = 277.52
Silhouette score= 0.85

```

▼ Example

```

1 # Load Iris dataset
2 iris = load_iris()
3 X_iris = iris.data
4 y_iris = iris.target
5 feature_names = iris.feature_names
6
7 df_iris = pd.DataFrame(X_iris, columns=feature_names)
8 df_iris['species'] = iris.target_names[y_iris]

```

```

1 # Standardize
2 scaler_iris = StandardScaler()
3 X_iris_scaled = scaler_iris.fit_transform(X_iris)
4
5 # Elbow method
6 k_range = range(1, 11)
7 wcss_iris = []
8 silhouette_score_iris = []
9 for k in k_range:
10     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
11     kmeans.fit(X_iris_scaled)
12     wcss_iris.append(float(np.round(kmeans.inertia_,2)))
13     if k==1:
14         silhouette_score_iris.append(0)
15     else:
16         silhouette_score_iris.append(float(silhouette_score(X_iris_scaled, kmeans.labels_).round(4)))
17
18 print("WCSS =\n",wcss_iris)
19 print("Silhoutte score =\n",silhouette_score_iris)

```

```

WCSS =
[600.0, 222.36, 139.82, 114.09, 90.93, 81.54, 72.63, 62.54, 55.12, 47.39]
Silhoutte score =
[0, 0.5818, 0.4599, 0.3869, 0.3459, 0.3171, 0.3202, 0.3387, 0.3424, 0.3518]

```

```

1 # Apply K-Means with k=3
2 kmeans_iris = KMeans(n_clusters=3, random_state=42, n_init=10)
3 clusters_iris = kmeans_iris.fit_predict(X_iris_scaled)

```

```

1 # Accuracy (with best label mapping)
2 from scipy.stats import mode
3 labels_mapped = np.zeros_like(clusters_iris)
4 for i in range(3):
5     mask = clusters_iris == i
6     labels_mapped[mask] = mode(y_iris[mask], keepdims=True)[0][0]
7
8 accuracy = np.mean(labels_mapped == y_iris)
9 print(f"\nClustering Accuracy (with best mapping): {accuracy*100:.1f}%")
10 print("\n→ K-Means successfully discovered the 3 species without labels!")

```

Clustering Accuracy (with best mapping): 83.3%

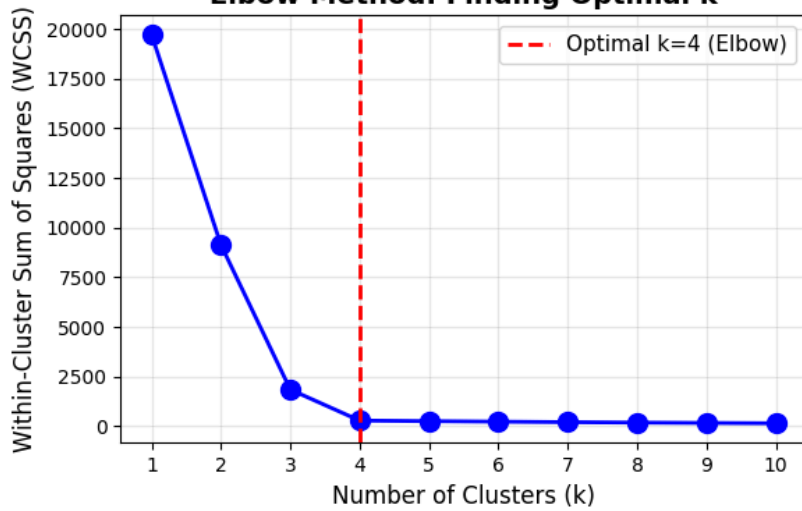
→ K-Means successfully discovered the 3 species without labels!

```

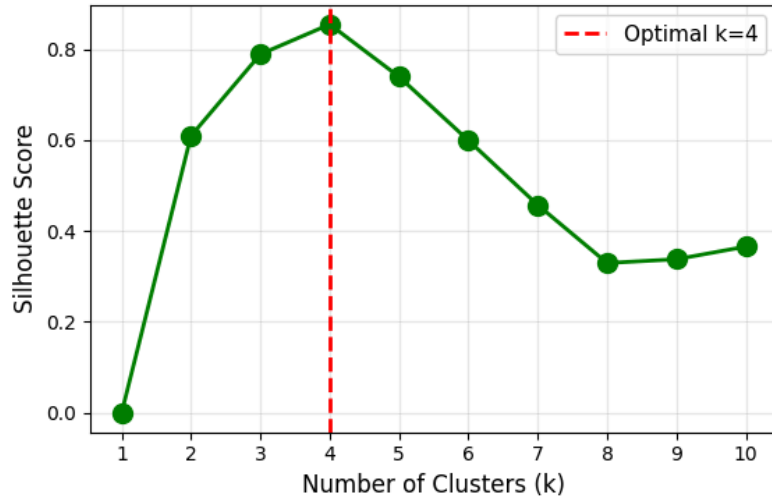
1 import matplotlib.pyplot as plt
2 # Generate data with clear clusters
3 X_elbow, y_elbow = make_blobs(n_samples=300, centers=4, n_features=2,
4                               cluster_std=0.7, random_state=42)
5
6 # Calculate WCSS for different k values
7 k_range = range(1, 11)
8 wcss = []
9 silhouette_scores = []
10
11 for k in k_range:
12     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
13     kmeans.fit(X_elbow)
14     wcss.append(kmeans.inertia_)
15
16     # Silhouette score (only for k >= 2)
17     if k >= 2:
18         silhouette_scores.append(silhouette_score(X_elbow, kmeans.labels_))
19     else:
20         silhouette_scores.append(0)
21
22 # Plot elbow curve
23 fig, axes = plt.subplots(2, 1, figsize=(6, 8))
24
25 # WCSS (Elbow) plot
26 axes[0].plot(k_range, wcss, 'bo-', linewidth=2, markersize=10)
27 axes[0].axvline(x=4, color='red', linestyle='--', linewidth=2, label='Optimal k=4 (Elbow)')
28 axes[0].set_xlabel('Number of Clusters (k)', fontsize=12)
29 axes[0].set_ylabel('Within-Cluster Sum of Squares (WCSS)', fontsize=12)
30 axes[0].set_title('Elbow Method: Finding Optimal k', fontsize=14, fontweight='bold')
31 axes[0].grid(True, alpha=0.3)
32 axes[0].legend(fontsize=11)
33 axes[0].set_xticks(k_range)
34
35 # Silhouette score plot
36 axes[1].plot(k_range, silhouette_scores, 'go-', linewidth=2, markersize=10)
37 axes[1].axvline(x=4, color='red', linestyle='--', linewidth=2, label='Optimal k=4')
38 axes[1].set_xlabel('Number of Clusters (k)', fontsize=12)
39 axes[1].set_ylabel('Silhouette Score', fontsize=12)
40 axes[1].set_title('Silhouette Score: Validating k', fontsize=14, fontweight='bold')
41 axes[1].grid(True, alpha=0.3)
42 axes[1].legend(fontsize=11)
43 axes[1].set_xticks(k_range)
44
45 plt.tight_layout()
46 plt.show()

```

Elbow Method: Finding Optimal k



Silhouette Score: Validating k



Heirarchical

```

1 from scipy.cluster.hierarchy import dendrogram, linkage
2 from sklearn.cluster import AgglomerativeClustering
3
4 # Generate sample data
5 np.random.seed(42)
6 X_hier = np.vstack([
7     np.random.randn(30, 2) + [2, 2],
8     np.random.randn(30, 2) + [8, 2],
9     np.random.randn(30, 2) + [5, 8]
10 ])
11
12 print(f"Generated {len(X_hier)} points for hierarchical clustering")
13
14 linkage_methods = ['single', 'complete', 'average', 'ward']
15 method = 'single'
16 linkage_matrix = linkage(X_hier, method=method)
17 # dendrogram(linkage_matrix)
18
19 linkage_methods = ['single', 'complete', 'average', 'ward']
20 method = 'single'
21 agg = AgglomerativeClustering(n_clusters=3, linkage=method)
22 labels = agg.fit_predict(X_hier)
23
24 k_range = range(2, 8)
25 silhouette_scores_hier = []
26
27 for k in k_range:
28     agg = AgglomerativeClustering(n_clusters=k, linkage='ward')
29     labels = agg.fit_predict(X_hier)
30     silhouette_scores_hier.append(silhouette_score(X_hier, labels))

```

Generated 90 points for hierarchical clustering

DBSCAN

```

1 from sklearn.cluster import DBSCAN
2 from sklearn.datasets import make_moons, make_circles
3
4 # Create datasets where K-Means fails
5 X_moons, _ = make_moons(n_samples=200, noise=0.05, random_state=42)
6 X_circles, _ = make_circles(n_samples=200, noise=0.05, factor=0.5, random_state=42)
7
8 print("Created challenging datasets for DBSCAN demonstration")

```

Created challenging datasets for DBSCAN demonstration

```

1 # Visualize DBSCAN concepts
2 from sklearn.neighbors import NearestNeighbors
3
4 # Create simple dataset
5 np.random.seed(42)
6 X_demo = np.random.randn(50, 2)
7
8 # Run DBSCAN
9 dbscan_demo = DBSCAN(eps=0.5, min_samples=5)
10 labels_demo = dbscan_demo.fit_predict(X_demo)
11
12 # Identify point types
13 core_samples = np.zeros(len(X_demo), dtype=bool)
14 core_samples[dbscan_demo.core_sample_indices_] = True
15 noise = labels_demo == -1
16 border = ~core_samples & ~noise
17
18 print(f"Core points: {core_samples.sum()}")
19 print(f"Border points: {border.sum()}")
20 print(f"Noise points: {noise.sum()}")

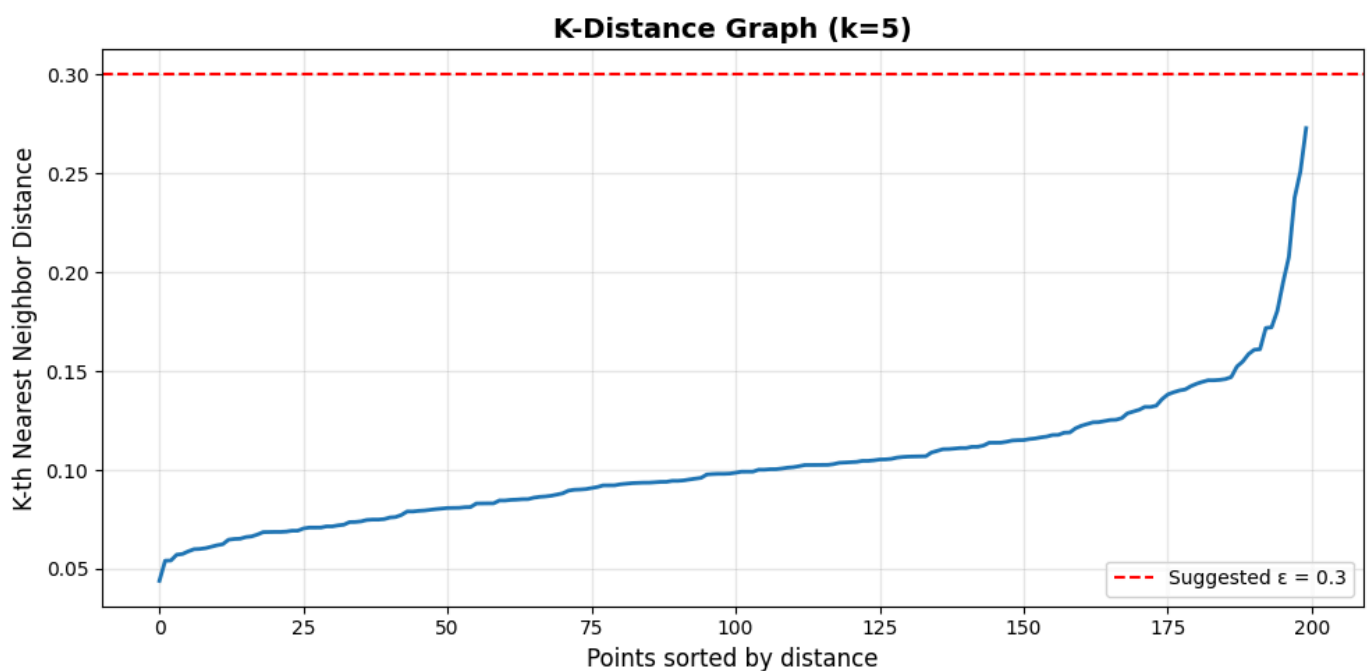
```

Core points: 17
Border points: 10
Noise points: 23

```

1 # K-distance graph for parameter selection
2 k = 5 # MinPts
3 nbrs = NearestNeighbors(n_neighbors=k).fit(X_moons)
4 distances, indices = nbrs.kneighbors(X_moons)
5 distances = np.sort(distances[:, k-1], axis=0)
6
7 plt.figure(figsize=(10, 5))
8 plt.plot(distances, linewidth=2)
9 plt.ylabel('K-th Nearest Neighbor Distance', fontsize=12)
10 plt.xlabel('Points sorted by distance', fontsize=12)
11 plt.title('K-Distance Graph (k=5)', fontsize=14, fontweight='bold')
12 plt.axhline(y=0.3, color='r', linestyle='--', label='Suggested  $\epsilon = 0.3$ ')
13 plt.grid(True, alpha=0.3)
14 plt.legend()
15 plt.tight_layout()
16 plt.show()
17
18 print("The elbow point suggests  $\epsilon \approx 0.3$ ")

```



The elbow point suggests $\epsilon \approx 0.3$

Cluster Quality Evaluation

Purity

```
1 from sklearn.datasets import load_iris
2
3 iris = load_iris()
4 X_iris = iris.data
5 y_true = iris.target
6
7 def purity_score(y_true, y_pred):
8     # Compute confusion matrix
9     contingency_matrix = np.zeros((len(np.unique(y_true)), len(np.unique(y_pred))))
10    for i, true_label in enumerate(np.unique(y_true)):
11        for j, pred_label in enumerate(np.unique(y_pred)):
12            contingency_matrix[i, j] = np.sum((y_true == true_label) & (y_pred == pred_label))
13
14    # Purity is sum of max values in each column divided by total
15    return np.sum(np.max(contingency_matrix, axis=0)) / np.sum(contingency_matrix)
16
17 # Test on Iris
18 kmeans_iris = KMeans(n_clusters=3, random_state=42, n_init=10)
19 labels_iris = kmeans_iris.fit_predict(X_iris)
20
21 purity = purity_score(y_true, labels_iris)
22
23 print(f"Purity Score: {purity:.3f}")
24 print("Higher purity means clusters are more homogeneous")
```

Purity Score: 0.893
Higher purity means clusters are more homogeneous

Entropy

```
1 def entropy_score(y_true, y_pred):
2     from scipy.stats import entropy as scipy_entropy
3
4     total_entropy = 0
5     n_samples = len(y_true)
6
7     for cluster_label in np.unique(y_pred):
8         cluster_mask = y_pred == cluster_label
9         cluster_size = np.sum(cluster_mask)
10
11        if cluster_size == 0:
12            continue
13
14        # Distribution of true labels in this cluster
15        label_counts = np.bincount(y_true[cluster_mask], minlength=len(np.unique(y_true)))
16        label_probs = label_counts / cluster_size
17
18        # Entropy of this cluster
19        cluster_entropy = scipy_entropy(label_probs, base=2)
20
21        # Weighted by cluster size
22        total_entropy += (cluster_size / n_samples) * cluster_entropy
23
24    return total_entropy
25
26 entropy = entropy_score(y_true, labels_iris)
27 print(f"Entropy Score: {entropy:.3f}")
28 print(f"Purity Score: {purity:.3f}")
29 print("\nLower entropy = better clustering")
30 print("Higher purity = better clustering")
```

Entropy Score: 0.394
Purity Score: 0.893

Lower entropy = better clustering
Higher purity = better clustering

RAND Index

```
1 from sklearn.metrics import rand_score, adjusted_rand_score
2
3 rand = rand_score(y_true, labels_iris)
```

```
4 adj_rand = adjusted_rand_score(y_true, labels_iris)
5
6 print(f"RAND Index: {rand:.3f}")
7 print(f"Adjusted RAND Index: {adj_rand:.3f}")
8 print("\nAdjusted RAND Index:")
9 print("• Range: [-1, 1]")
10 print("• 1: Perfect match")
11 print("• 0: Random labeling")
12 print("• Negative: Worse than random")
```

RAND Index: 0.880
Adjusted RAND Index: 0.730

Adjusted RAND Index:

- Range: [-1, 1]
- 1: Perfect match
- 0: Random labeling
- Negative: Worse than random

▼ Silhouette Score

```
1 from sklearn.metrics import silhouette_samples
2
3 # Compute silhouette for each sample
4 silhouette_vals = silhouette_samples(X_iris, labels_iris)
5 silhouette_avg = np.mean(silhouette_vals)
6
7 print(f"Average Silhouette Score: {silhouette_avg:.3f}")
8 print("Green points: Well-clustered")
9 print("Yellow/Red points: Poorly clustered or misassigned")
```

Average Silhouette Score: 0.553
Green points: Well-clustered
Yellow/Red points: Poorly clustered or misassigned

▼ Comprehensive Comparison

```
1 # Compute all metrics
2 from sklearn.metrics import adjusted_mutual_info_score, fowlkes_mallows_score
3
4 metrics = {
5     'Purity': purity,
6     'Entropy': entropy,
7     'RAND Index': rand,
8     'Adjusted RAND': adj_rand,
9     'Silhouette': silhouette_avg,
10    'Adj. Mutual Info': adjusted_mutual_info_score(y_true, labels_iris),
11    'Fowlkes-Mallows': fowlkes_mallows_score(y_true, labels_iris)
12 }
13
14 # Display as table
15 print("\nCluster Quality Metrics for K-Means on Iris:")
16 print("=" * 44)
17 for metric, value in metrics.items():
18     print(f"|{' '*7}{metric:20s}: {value:6.3f} |{' '*6}|")
19 print("=" * 44)
20
21 print("\nInterpretation:")
22 print("• Purity, RAND, Silhouette, AMI, FM: Higher is better")
23 print("• Entropy: Lower is better")
24 print("• Adjusted metrics correct for chance")
25
```

Cluster Quality Metrics for K-Means on Iris:

```
=====
|      Purity           :   0.893   |
|      Entropy          :   0.394   |
|      RAND Index       :   0.880   |
|      Adjusted RAND    :   0.730   |
|      Silhouette       :   0.553   |
|      Adj. Mutual Info :   0.755   |
|      Fowlkes-Mallows  :   0.821   |
=====
```

Interpretation:

- Purity, RAND, Silhouette, AMI, FM: Higher is better
- Entropy: Lower is better
- Adjusted metrics correct for chance

Algorithm Comparison Summary

Algorithm	Cluster Shape	Requires k	Handles Noise	Time Complexity	Best For
K-Means	Spherical	Yes	No	$O(nkt)$	Large datasets, spherical clusters
Hierarchical	Any	No	No	$O(n^3 \log n)$	Small datasets, hierarchy needed
DBSCAN	Arbitrary	No	Yes	$O(n \log n)$	Arbitrary shapes, noise detection
Spectral	Non-convex	Yes	No	$O(n^3)$	Graph data, complex shapes

Decision Guide

Use K-Means when:

- Clusters are roughly spherical
- You know k in advance
- You have large datasets
- Speed is important

Use Hierarchical when:

- You need a hierarchy of clusters
- You don't know k
- Dataset is small (< 10,000 points)
- You want to explore different k values

Use DBSCAN when:

- Clusters have arbitrary shapes
- You need to identify outliers
- Cluster density is roughly uniform
- You don't know k

Use Spectral when:

- Data has graph structure
- Clusters are non-convex
- You can afford computational cost
- K-Means fails on your data

8. Regression

Simple Linear Regression

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, r2_score

```

```

1 # Generate sample data for simple linear regression
2 np.random.seed(42)
3 X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
4 y = 2 + 1.5 * X.flatten() + np.random.normal(0, 1, 10)
5
6 # Create and fit the model
7 model = LinearRegression()
8 model.fit(X, y)
9
10 # Get predictions
11 y_pred = model.predict(X)

```

Multi Linear Regression

```

1 # Sample data with two independent variables
2 np.random.seed(42)
3 X_multi = np.array([
4     [1, 1], [1, 2], [2, 2], [2, 3], [3, 3],
5     [3, 4], [4, 4], [4, 5], [5, 5], [5, 6]
6 ])
7 y_multi = 1 + 2*X_multi[:, 0] + 1.5*X_multi[:, 1] + np.random.normal(0, 0.5, 10)
8
9 # Create and fit the model
10 model_multi = LinearRegression()
11 model_multi.fit(X_multi, y_multi)
12
13 # Get predictions
14 y_pred_multi = model_multi.predict(X_multi)
15
16 # Calculate metrics

```

```
17 mse = mean_squared_error(y_multi, y_pred_multi)
18 w2 = w2 + lr * (y_multi - y_pred_multi)
```

✓ OLS

```
1 from numpy.linalg import inv
2
3 # Generate sample data
4 np.random.seed(42)
5 n_samples = 20
6 X = np.linspace(0, 10, n_samples).reshape(-1, 1)
7 y = 2 + 1.5 * X.flatten() + np.random.normal(0, 1.5, n_samples)
8
9 # Add bias term (column of ones)
10 X_with_bias = np.hstack([np.ones((n_samples, 1)), X])
11
12 # OLS solution using normal equation
13 # w_hat = (X^T X)^-1 X^T y
14 XTX = X_with_bias.T @ X_with_bias
15 XTy = X_with_bias.T @ y
16 w_hat = inv(XTX) @ XTy
17
18 print("OLS Solution using Normal Equation:")
19 print(f"Intercept (w_0): {w_hat[0]:.4f}")
20 print(f"Slope (w_1): {w_hat[1]:.4f}")
21
22 # Calculate predictions and residuals
23 y_pred = X_with_bias @ w_hat
24 residuals = y - y_pred
25 ssr = np.sum(residuals**2)
26
27 print(f"\nSum of Squared Residuals: {ssr:.4f}")
28 print(f"Mean Squared Error: {ssr/n_samples:.4f}")
29
30 # Compare with scikit-learn
31 from sklearn.linear_model import LinearRegression
32 model_sklearn = LinearRegression()
33 model_sklearn.fit(X, y)
34
35 print(f"\nComparison with scikit-learn:")
36 print(f"Intercept (sklearn): {model_sklearn.intercept_:.4f}")
37 print(f"Slope (sklearn): {model_sklearn.coef_[0]:.4f}")
```

✓ Linear Algebraic Interpretation

```
1 # Create sample data
2 n_samples = 15
3 X = np.linspace(0, 10, n_samples).reshape(-1, 1)
4 y = 2 + 1.5 * X.flatten() + np.random.normal(0, 1.5, n_samples)
5
6 # Add bias term
7 X_with_bias = np.hstack([np.ones((n_samples, 1)), X])
8
9 # OLS solution
10 from numpy.linalg import inv
11 w_hat = inv(X_with_bias.T @ X_with_bias) @ X_with_bias.T @ y
12 y_pred = X_with_bias @ w_hat
13 residuals = y - y_pred
14
15 print("Linear Algebraic Interpretation of OLS:")
16 print("=" * 50)
17 print(f"\nDesign Matrix X* shape: {X_with_bias.shape}")
18 print(f"Observation vector y shape: {y.shape}")
19 print(f"Predicted vector ŷ shape: {y_pred.shape}")
20 print(f"Residual vector e shape: {residuals.shape}")
21
22 # Verify orthogonality: X^T e should be approximately zero
23 orthogonality_check = X_with_bias.T @ residuals
24 print(f"\nOrthogonality Check (X^T e):")
25 print(f" {orthogonality_check}")
26 print(f" (Should be close to zero, max value: {np.max(np.abs(orthogonality_check)):.2e})")
27
28 # Calculate projection matrix
29 P = X_with_bias @ inv(X_with_bias.T @ X_with_bias) @ X_with_bias.T
30 print(f"\nProjection Matrix P shape: {P.shape}")
31 print(f"Predicted values ŷ = P y")
32 print(f"Verification: ||P y - ŷ|| = {np.linalg.norm(P @ y - y_pred):.2e}")
```

Linear Algebraic Interpretation of OLS:

=====

```

Design Matrix X* shape: (15, 2)
Observation vector y shape: (15,)
Predicted vector  $\hat{y}$  shape: (15,)
Residual vector e shape: (15,)

Orthogonality Check ( $X^T e$ ):
[ 4.88498131e-15 -7.81597009e-14]
(Should be close to zero, max value: 7.82e-14)

Projection Matrix P shape: (15, 15)
Predicted values  $\hat{y} = P y$ 
Verification:  $\|P y - \hat{y}\| = 7.35e-15$ 

```

Polynomial Regression

```

1 from sklearn.preprocessing import PolynomialFeatures
2 from sklearn.linear_model import LinearRegression
3 from sklearn.metrics import mean_squared_error, r2_score
4
5 # Generate non-linear sample data
6 np.random.seed(42)
7 n_samples = 50
8 X = np.linspace(-5, 5, n_samples).reshape(-1, 1)
9 y = 2 + 0.5*X.flatten() + 0.3*X.flatten()**2 - 0.1*X.flatten()**3 + np.random.normal(0, 2, n_samples)
10
11 # Create polynomial features for different degrees
12 degrees = [1, 2, 3, 5]
13 models = {}
14 predictions = {}
15
16
17 # Generate smooth curve for plotting
18 X_smooth = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)
19
20 for idx, degree in enumerate(degrees):
21     # Create polynomial features
22     poly_features = PolynomialFeatures(degree=degree)
23     X_poly = poly_features.fit_transform(X)
24     X_smooth_poly = poly_features.transform(X_smooth)
25
26     # Fit model
27     model = LinearRegression()
28     model.fit(X_poly, y)
29
30     # Make predictions
31     y_pred = model.predict(X_poly)
32     y_smooth = model.predict(X_smooth_poly)
33
34     # Calculate metrics
35     mse = mean_squared_error(y, y_pred)
36     r2 = r2_score(y, y_pred)
37
38     models[degree] = model
39     predictions[degree] = y_pred
40
41 # Comparison of all models
42 print("\n" + "="*60)
43 print("Model Comparison:")
44 print("="*60)
45 for degree in degrees:
46     poly_features = PolynomialFeatures(degree=degree)
47     X_poly = poly_features.fit_transform(X)
48     y_pred = models[degree].predict(X_poly)
49     mse = mean_squared_error(y, y_pred)
50     r2 = r2_score(y, y_pred)
51     print(f"Degree {degree}: MSE = {mse:.4f}, R² = {r2:.4f}")

```

```

=====
Model Comparison:
=====
Degree 1: MSE = 15.7035, R² = 0.4332
Degree 2: MSE = 8.6384, R² = 0.6882
Degree 3: MSE = 3.0612, R² = 0.8895
Degree 5: MSE = 3.0071, R² = 0.8915

```

Logarithmic Regression

```

1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_squared_error, r2_score
3
4 # Generate logarithmic sample data
5 np.random.seed(42)

```

```

5 np.random.seed(42)
6 n_samples = 50
7 X = np.linspace(0.5, 10, n_samples).reshape(-1, 1)
8 # True relationship: y = 3 + 2*log(x)
9 y = 3 + 2*np.log(X.flatten()) + np.random.normal(0, 0.3, n_samples)
10
11 # Method 1: Simple Logarithmic Regression
12 X_log = np.log(X)
13 model_log = LinearRegression()
14 model_log.fit(X_log, y)
15 y_pred_log = model_log.predict(X_log)
16
17 # Calculate metrics
18 mse_log = mean_squared_error(y, y_pred_log)
19 r2_log = r2_score(y, y_pred_log)
20
21 print("Logarithmic Regression Results:")
22 print("="*50)
23 print(f"Model: y = {model_log.intercept_:.4f} + {model_log.coef_[0]:.4f} * log(x)")
24 print(f"MSE: {mse_log:.4f}")
25 print(f"R² Score: {r2_log:.4f}")
26
27 # For comparison: fit a polynomial to log-transformed data
28 from sklearn.preprocessing import PolynomialFeatures
29
30 poly_features = PolynomialFeatures(degree=2)
31 X_log_poly = poly_features.fit_transform(X_log)
32 model_log_poly = LinearRegression()
33 model_log_poly.fit(X_log_poly, y)
34 y_pred_log_poly = model_log_poly.predict(X_log_poly)
35
36 mse_log_poly = mean_squared_error(y, y_pred_log_poly)
37 r2_log_poly = r2_score(y, y_pred_log_poly)
38
39 print(f"\nQuadratic Logarithmic Regression:")
40 print(f"Model: y = {model_log_poly.intercept_:.4f} + {model_log_poly.coef_[1]:.4f} * log(x) + {model_log_poly.coef_[2]:.4f} * log(x)²")
41 print(f"MSE: {mse_log_poly:.4f}")
42 print(f"R² Score: {r2_log_poly:.4f}")

```

```

Logarithmic Regression Results:
=====
Model: y = 3.0659 + 1.9077 * log(x)
MSE: 0.0722
R² Score: 0.9651

Quadratic Logarithmic Regression:
Model: y = 3.0882 + 1.7948 * log(x) + 0.0534 * log(x)²
MSE: 0.0711
R² Score: 0.9656

```

9. Imports & Key Syntaxes

Imports

```

1 # Libraries needed for DAL Mid-term
2 # -----
3
4 # General Libraries
5 import numpy as np
6 import pandas as pd
7
8 # Probability and Statistics
9 from scipy import stats
10 from scipy.special import comb, perm, factorial
11 from itertools import permutations, combinations, combinations_with_replacement
12
13 # Optimization Methods
14 from scipy.optimize import minimize, Bounds, linprog, minimize_scalar
15
16 # PCA
17 from sklearn.decomposition import PCA
18 from sklearn.preprocessing import StandardScaler
19
20 # Clustering
21 ## Datasets
22 from sklearn.datasets import load_iris, load_digits, load_wine
23 from sklearn.datasets import make_blobs, make_moons, make_circles, make_classification
24 ## Clustering Algorithms
25 from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
26 from scipy.cluster.hierarchy import dendrogram, linkage

```

```

27 ## Data Preprocessing
28 from sklearn.preprocessing import StandardScaler
29 ## Cluster Quality Evaluation metrics
30 from sklearn.metrics import silhouette_score, rand_score, adjusted_rand_score, mean_squared_error, r2_score
31
32 # Regression

```

▼ Syntaxes

```

1 # Hypothesis Tests
2 mu_0 = 120 ## Hypothesized mean
3 ## T-test (two tailed)
4 t_statistic, p_value_two_sided = stats.ttest_1samp(bulb_lifetimes, mu_0, alternative='two-sided')
5 ## T-test (Right tailed)
6 t_statistic, p_value_right = stats.ttest_1samp(new_method_scores, mu_0, alternative='greater')
7 ## T-test (Left tailed)
8 t_statistic, p_value_left = stats.ttest_1samp(new_process_times, mu_0, alternative='less')
9
10 # Optimization (Minimization)
11 ## Univariate
12 result = minimize_scalar(lambda x: -profit_function(x), bounds=(0, 5000), method='bounded')
13 x_optimal_numerical = result.x
14 f_optimal_numerical = -result.fun
15
16 ## Multivariate
17 result = minimize(lambda x: -profit_2d(x), x0=np.array([0, 0]), method='BFGS')
18 x_optimal_numerical = result.x
19 f_optimal_numerical = -result.fun
20
21 ## Linear Programming
22 ### Coefficients
23 c = np.array([3, 2]) ### Objective coefficients (we'll minimize -c for maximization)
24 A_ub = np.array([[2, 1], [1, 1]]) ### Inequality constraint matrix
25 b_ub = np.array([100, 80]) ### Inequality constraint bounds
26 x_bounds = [(0, None), (0, None)] ### Non-negativity constraints
27 ### Solve using linprog (minimizes c^T x, so we negate c)
28 result_lp = linprog(-c, A_ub=A_ub, b_ub=b_ub, bounds=x_bounds, method='highs')
29
30 ## Quadratic Programming
31 constraints = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1})
32 bounds = [(0, 1) for _ in range(3)]
33 x0 = np.array([0.33, 0.33, 0.34])
34 result_qp = minimize(qp_objective, x0, method='SLSQP', bounds=bounds, constraints=constraints)
35
36 ## Non-Linear Programming
37 constraints = {'type': 'eq', 'fun': nlp_constraint}
38 initial_points = [np.array([1.0, 1.0]), np.array([0.5, 1.5]), np.array([-1.0, -1.0])]
39 result = minimize(nlp_objective, x0, method='SLSQP', constraints=constraints)
40
41 # PCA
42 scaler = StandardScaler()
43 X_iris_scaled = scaler.fit_transform(X_iris)
44 pca_sklearn = PCA(n_components=2)
45 X_iris_pca = pca_sklearn.fit_transform(X_iris_scaled)
46
47 # KMeans
48 scaler = StandardScaler()
49 X_scaled = scaler.fit_transform(x)
50 kmeans = KMeans(n_clusters=2, n_init=10, random_state=42)
51 kmeans.fit(X_scaled)
52 labels = kmeans.labels_ # (or) labels = kmeans.fit_predict(X_scaled)
53 wcss = kmeans.inertia_
54 silhouettescore = silhouette_score(X_scaled, kmeans.labels_)
55
56 # Dendrogram
57 linkage_methods = ['single', 'complete', 'average', 'ward']
58 linkage_matrix = linkage(X_hier, method=linkage_methods[0])
59 dendrogram(linkage_matrix) # This returns the dendrogram chart
60
61 # Agglomerative
62 linkage_methods = ['single', 'complete', 'average', 'ward']
63 agg = AgglomerativeClustering(n_clusters=3, linkage=linkage_methods[0])
64 labels = agg.fit_predict(X_hier)
65
66 # DBSCAN
67 dbscan_demo = DBSCAN(eps=0.5, min_samples=5)
68 labels_demo = dbscan_demo.fit_predict(X_demo)
69 core_samples = np.zeros(len(X_demo), dtype=bool)
70 core_samples[dbscan_demo.core_sample_indices_] = True
71 noise = labels_demo == -1

```

```

72 border = ~core_samples & ~noise
73
74 # Cluster Quality Evaluation techniques
75 def purity_score(y_true, y_pred):
76     ## Compute confusion matrix
77     contingency_matrix = np.zeros((len(np.unique(y_true)), len(np.unique(y_pred))))
78     for i, true_label in enumerate(np.unique(y_true)):
79         for j, pred_label in enumerate(np.unique(y_pred)):
80             contingency_matrix[i, j] = np.sum((y_true == true_label) & (y_pred == pred_label))
81     ## Purity is sum of max values in each column divided by total
82     return np.sum(np.max(contingency_matrix, axis=0))/np.sum(contingency_matrix)
83
84 purity = purity_score(y_true, labels_iris)
85
86 def entropy_score(y_true, y_pred):
87     from scipy.stats import entropy as scipy_entropy
88     total_entropy = 0
89     n_samples = len(y_true)
90     for cluster_label in np.unique(y_pred):
91         cluster_mask = y_pred == cluster_label
92         cluster_size = np.sum(cluster_mask)
93         if cluster_size == 0:
94             continue
95         # Distribution of true labels in this cluster
96         label_counts = np.bincount(y_true[cluster_mask], minlength=len(np.unique(y_true)))
97         label_probs = label_counts / cluster_size
98         # Entropy of this cluster
99         cluster_entropy = scipy_entropy(label_probs, base=2)
100        # Weighted by cluster size
101        total_entropy += (cluster_size / n_samples) * cluster_entropy
102    return total_entropy
103
104 entropy = entropy_score(y_true, labels_iris)
105
106 rand = rand_score(y_true, labels_iris)
107 adj_rand = adjusted_rand_score(y_true, labels_iris)
108
109 silhouette_vals = silhouette_samples(X_iris, labels_iris)
110 silhouette_avg = np.mean(silhouette_vals)
111
112
113 ## Linear Regression
114 # Create and fit the model

```